

Loading Screen Pro

User Manual – Version 0.5

Loading Screen Pro is an asset for Unity 3d to quickly add a good looking loading screen to your game. All you need to do is decide on one of the loading scenes included in the asset or create one yourself. Then it's just a matter of exchanging the basic Unity method call to load a new scene (different based on your version of Unity) with the `LoadingScreenPro.LoadScene()`.

A promotional image for Loading Screen Pro. The top section has a blue background with the product name 'Loading Screen PRO' in a stylized font and the logo for Martin Nerurkar Playful Systems. The middle section features a list of features in orange text on a dark background. To the right, there are three overlapping screenshots of different loading screens: one with a forest scene and a 0% progress bar, one with a sunset scene and a progress bar, and one with a dark scene and a progress bar. A large blue speech bubble contains the text 'INCLUDES HEALTH & PROGRESS BAR PRO!'. At the bottom, it says 'using Unity UI, source included!' in orange text.

Loading Screen PRO

Martin Nerurkar
Playful Systems

- One line integration
- Display image & text based on target scene
- Show random game tips
- 8 Example loading scenes
- Easily extendable

INCLUDES HEALTH & PROGRESS BAR PRO!

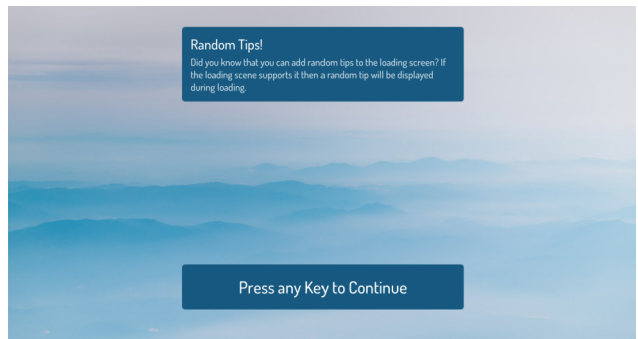
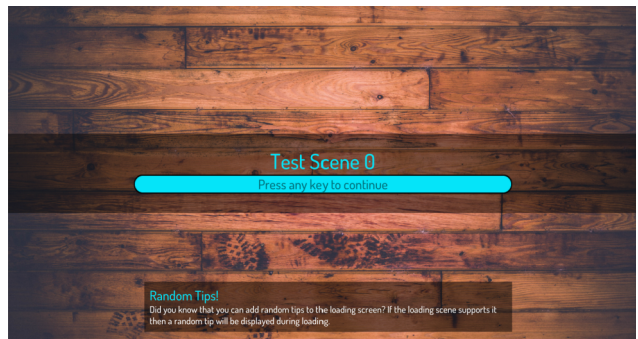
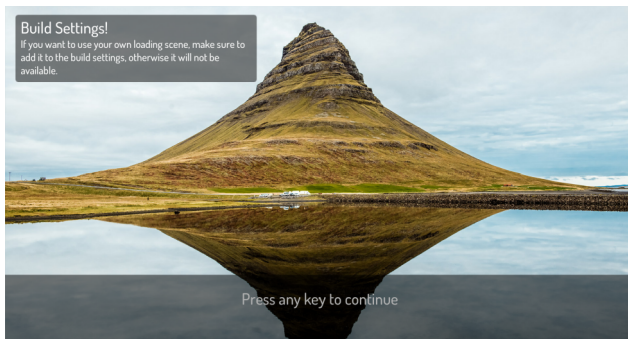
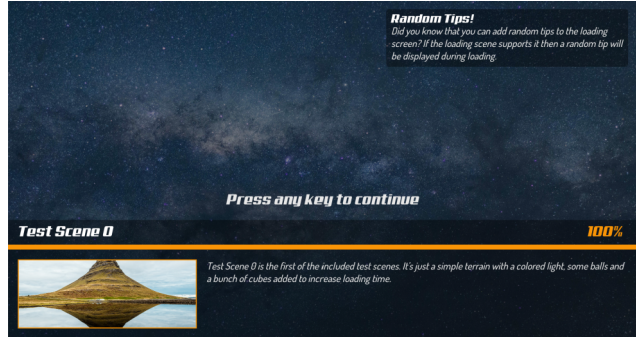
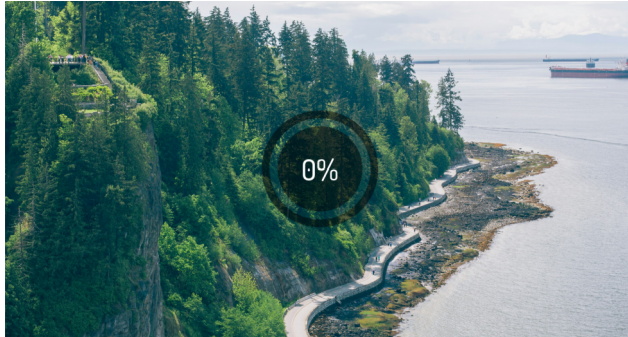
using **Unity UI**, source included!

Features

- One line integration
- Display name, description and image preview based on scene that's being loaded
- Display random game tips
- Easily extendable with your own functionality

Example Scenes

A number of pre-designed loading scenes are part of the package. These are complete and can be used in a wide variety of projects. Here's an overview:



More options will be added with future versions of this asset.

Using the Asset

Here's what you need to do to use Loading Screen Pro:

1. Select a scene from those included or create your own
2. Add your intended scene to the Unity build settings (there's a handy button on the LoadingScreenPro inspector that takes care of that for you)
3. Edit a single line in the LoadingScreenConfig script to the name of your loading screen
4. Replace all calls to load a scene in your game with **LoadingScreenPro.LoadScene(string)** or **LoadingScreenPro.LoadScene(int)**

This last step is different based on your version of Unity. The original Unity method call you used to load levels will have to be replaced by the method call above. The method you will have to replace is either going to be:

Unity >= 5.3	Application.LoadLevel()	Application.LoadLevelAsync()
Unity < 5.3	SceneManager.LoadScene()	SceneManager.LoadSceneAsync()

Changing the Loading Screen Config

The Loading Screen Config is an asset that can be found in the PS-LoadingScreen folder. Select it and edit its info in the inspector.

Loading Behavior

- **Load Additively**
This determines whether the scene is loaded in the background additively or not. If it's loaded additively both the loading scene and the new scene will be actively briefly before the loading scene is then disabled
- **Load Thread Priority**
Lower priority means a background operation will run less often and will take up less time, but will progress more slowly.

Scene Infos

Scene Info can be used if you want the loading screen to display specific graphics or text based on the scene that's being loaded. Note that the loading screen you're using needs to support this or nothing will be displayed. Some of the given loading screens only support the loading image, while others support only the text. You can also chose to only use some of these fields.

- **Scene Name**
This has to be the name of the scene that the following information belong to. It is used by the Loading Screen to find out which set of info to display
- **Image Name**
The name of the sprite that should be shown for this scene. Note that all images are loaded from "Resources/ScenePreviews/".
- **Header**
This is the first string that can be displayed for the loaded scene. It's generally used as

the name of the scene that's being loaded.

- **Description**

This is the second string. It's generally used as a longer string that describes the scene

Game Tips

Game Tips are randomly picked when loading a scene. As with the Scene Infos you can choose to use these fields as you want, even leaving them empty. However make sure your scene supports the game tips.

- **Header**

This is the first string that can be displayed. It's generally used as the name of the tip.

- **Description**

This is the second string. It's generally used to actually give the tip.

Building your own Loading Scene

At the heart of your own design is the LoadingScreenPro script. This needs to be on an object in the scene. This controls the loading and displays current loading state. Most of the fields here are optional.

- **Config**

This needs to refer to the config asset found in the PS-LoadingScreen folder. If it's not set there's a convenience button that automatically searches for that asset for you.

- **Audio Listener**

This needs to point to the audio listener of the camera in this scene. This is needed only if you load the scene additively to ensure only one audio listener is active.

- **Behavior After Load**

This allows you determine whether the loading screen closes automatically or waits for input from the player.

- **Time To Auto Continue**

This is used when closing the screen automatically to add a short delay before activating the loaded scene. When waiting for player input it's not used.

- **Scene Info Header**

This refers to a text component that is used to display the header field from the Scene Info..

- **Scene Info Description**

Used to display the description field from the Scene Info..

- **Scene Info Image**

Refers to an image component. Used to display the image found with the image name given in the scene info..

- **Tip Header**

A text component that displays the header field from the picked Game Tip..

- **Tip Description**

A text component that displays the description of the Game Tip.

- **Fade Settings (doFade, durations & colors)**

These settings determine whether and how the loading screen will fade into. Note that

the target scene needs to deal with fading out from that color by itself.

- **Loading Canvas Group**
This refers to a canvas group that will be hidden after the loading is done.
- **ProgressBar**
This refers to a ProgressBarPro component that will be used to show progress.
- **Loading Bar**
This refers to an image component set to fillable. It will be filled as the target scene loads. This is a simple alternative to the ProgressBarPro.
- **Loading Text**
A text component that displays the current loading state. This is a simple alternative to the ProgressBarPro.
- **Loading String**
This formats the text that is put into Loading Text. Writing "#progress#" will replace that with a value from 0 to 100.
- **Confirmation Canvas Group**
A canvas group that will be shown once the loading is done and the screen asks for input from the player.

Extending the Loading Screen

If you want to extend the loading screen functionality there are two main ways to do this:

Controlling visuals your way

If you want to react differently to the loading progress you can inherit from LoadingScreenBase and override the virtual methods to take control of the display by yourself. This allows you to add custom components such as your own progress bar, use animations etc.

Different Game Tips / Scene Infos

For example you want to ensure that tips are selected based on the players current state. Or you have your tips in an external xml file.

This can easily done by creating new classes that inherit from LoadingScreenConfig, SceneInfo and GameTip.

For example you can create your own data structures on top of the base classes for ScenInfo and GameTip. Note that if you've changed the data structures your new config you will also need a new LoadingScreenBase that can work with the new information properly

Alternatively you can override the GetSceneInfo() and GetGameTip() methods in your new Config with your own methods or add new methods. This allows you to control which tips are displayed and when.

Credits

All background artwork is taken from unsplash.com

Aero font is from www.be.net/nirmalbiswas

Dosis font is from Edgar Tolentino and Pablo Impallari (www.impallari.com|impallari@gmail.com), Iginio Marini. (www.ikern.com|mail@iginomarini.com).